FLA LAB PROJECT

Viva & Project Preparation Guide

DFA Simulation • NFA to DFA • DFA Minimization • DFA Intersection • PDA Simulation

All 5 programs compiled, run and verified • 68 theory questions • Java basics with code lines

PART A - Project overview & how to run

PART B - Java basics viva (static, data structures, core Java)

PART C - Automata theory viva - 68 questions in 3 levels

PART D - One-page cheat sheet for the last minute

Formal Languages & Automata | October 2026

PART A - PROJECT OVERVIEW & HOW TO RUN

A1. The five programs

#	Program	What it does	Technique / algorithm	Key data structures
1	**DFA Simulation**	Reads a DFA and tests whether input strings are accepted or rejected; prints the state path	Direct simulation of δ*	char[] alphabet, int[][] transition table, boolean[] final states, StringBuilder
2	**NFA to DFA**	Converts an NFA (epsilon moves optional) into an equivalent DFA	Subset construction + ε-closure	List<Integer>[][] table, TreeSet, HashMap, Queue (BFS), ArrayDeque stack (DFS)
3	**DFA Minimi-zation**	Reduces a DFA to the fewest states that accept the same language	Remove unreachable states (BFS) + table filling (Myhill-Nerode)	int[][] delta, boolean[][] marked pairs, Queue, int[] group mapping
4	**DFA Inter-section**	Builds a DFA for L1 ∩ L2 and tests strings on both machines	Product construction (reachable pairs only)	static class DFA, ArrayList<int[]>, HashMap<String,Integer>, Queue
5	**PDA Simulation**	Simulates a (non-deterministic) pushdown automaton, shows every stack operation and the accepting path	BFS over configurations (IDs) with parent pointers	static classes Rule & Config, String as stack (top = index 0), Queue, HashSet seen

Folder layout: for_submission/ = clean copy for sir, with_comments/ = same code with every line explained. Both compile and give identical output.

A2. Compile and run (Java 21 is already installed on this laptop)

```
cd "C:\Users\bsain\Desktop\AI\SEM 5\FLA\FLA_Project\for_submission"

javac *.java                        (no output = success)

java DFASimulation                  (class name only - no .java)

java NFAtoDFA | java DFAMinimization | java DFAIntersection | java PDASimulation
```

Common error	Meaning / fix
`ArrayIndexOutOfBounds Exception`	State number ≥ number of states (states start from 0)
`InputMismatch Exception`	Typed text where a number was expected - rerun and follow the prompt
`Could not find or load main class`	Wrong folder, or you typed java X.java - drop the .java
`'javac' is not recognized`	Old Command Prompt window - open a NEW one after installing Java
PDA: symbol 'e'	'e' is reserved for epsilon - never use it as an input symbol

A3. Tested sample inputs (type in this exact order)

1. DFA - strings ending in "ab" (3 states, symbols a b)

```
3        <- number of states        0      <- start state

2        <- number of symbols       1      <- number of final states

a b      <- the symbols             2      <- final state

1 0      <- row q0:  d(q0,a)=q1  d(q0,b)=q0

1 2      <- row q1:  d(q1,a)=q1  d(q1,b)=q2

1 0      <- row q2:  d(q2,a)=q1  d(q2,b)=q0
```

Then type strings: ab -> ACCEPTED, aab -> ACCEPTED, ba -> REJECTED, eps -> REJECTED, exit

2. NFA to DFA - strings ending in "01" (no epsilon)

```
3      2      0 1      n

2 0 1   <- d(q0,0)={q0,q1}      1 0    <- d(q0,1)={q0}

0       <- d(q1,0)={}           1 2    <- d(q1,1)={q2}

0       <- d(q2,0)={}           0      <- d(q2,1)={}

0       <- start                1  2   <- one final state: q2
```

*Output: 3 DFA states -> D0={q0}, D1={q0,q1}, *D2={q0,q2} (final)*

3. DFA Minimization - "ends with abb" + 1 unreachable state

```
6      2      a b

1 2    <- q0     1 3    <- q1     1 2    <- q2

1 4    <- q3     1 2    <- q4     0 5    <- q5 (unreachable)

0      <- start         1  4    <- final state q4
```

Output: q5 removed, q0 and q2 merged -> minimized DFA has 4 states

4. Intersection - L1 = even number of a's, L2 = ends with b

```
2  a b                          <- symbols (same for both DFAs)

DFA1: 2 / 1 0 / 0 1 / 0 / 1 / 0      (start 0, final q0)

DFA2: 2 / 0 1 / 0 1 / 0 / 1 / 1      (start 0, final q1)

then test:  aab  -> ACCEPTED          ab  -> REJECTED     exit
```

Output: 4 product states P0..P3; P2=(p0,q1) is the only final pair

5. PDA - $a^n b^n$ accepted by EMPTY STACK

```
2      0      Z      2      5

0 a Z 0 AZ      <- on a, push A over Z

0 a A 0 AA      <- on a, push another A

0 b A 1 e       <- first b: pop A, go to q1

1 b A 1 e       <- every next b: pop A

1 e Z 1 e       <- input done: pop Z -> stack empty -> accept

then test:  aabb  -> ACCEPTED (full stack trace printed)   aab -> REJECTED   exit
```

Bonus (non-deterministic, accept by final state): even palindromes ww^R - 12 rules, input abba -> ACCEPTED, aba -> REJECTED (full inputs in HOW_TO_RUN.md)

PART B - JAVA BASICS VIVA (with lines from YOUR code)

Every answer below points to a real line in your program, so you can show the code while answering.

B1. "What is static?" - the question they love

One-line answer: static means the member belongs to the **class itself**, not to any single object. One copy is created when the class is loaded and it is shared by the whole program; it can be used without creating an object.

Static variable (class variable)

```
static int n;                 // NFAtoDFA.java - top of class

static List<Rule> rules;      // PDASimulation.java

static char[] symbols;        // DFAIntersection.java
```

One copy exists for the whole class. If 100 objects were created they would all share the same n. The opposite is an **instance variable** (no static) - each object gets its own copy. Used here because many methods (main, closure, simulate, readDFA) need these values without passing them as parameters.

Static method

```
static int indexOf(char[] symbols, char c)     // DFASimulation.java

static Set<Integer> closure(Set<Integer> s)    // NFAtoDFA.java

static boolean accepts(Config c, String w)     // PDASimulation.java
```

Called directly on the class, no object needed: closure(startSet). **Rule they may ask:** a static method cannot use instance variables or this, because there is no object - it can only use static data and its own parameters.

public static void main(String[] args) - explain every word

Word	Meaning
public	visible everywhere; the JVM must be able to call it from outside
static	the JVM runs it without creating an object of your class
void	returns nothing
main	fixed name the JVM looks for as the entry point
String[] args	command-line arguments as an array of Strings

Follow-up: "Why is main static?" If main were non-static, the JVM would first have to create an object using a constructor - but calling a constructor already needs a running program, a circular problem. Static removes the need for an object, so the JVM can start the program directly. **Static block (bonus):** static { ... } runs once when the class is loaded, before main - not used in my programs, but I know it exists.

B2. "What data structures are we using?" - master answer

Say first: "Mostly arrays for the automaton tables, and Java collection classes (ArrayList, HashMap, HashSet, TreeSet, Queue, Stack) for the algorithms."

Program	Structure	Exact use	Why
DFA Sim	char[] / int[][] / boolean[]	alphabet, delta table, isFinal flags	fixed size, O(1) access: delta[state][symbol] = next state

Program	Structure	Exact use	Why
DFA Sim	StringBuilder	builds the path text	faster than + inside a loop
NFA→DFA	List<Integer>[][]	NFA transition table	one cell = a SET of next states - that is the whole NFA vs DFA difference
NFA→DFA	TreeSet<Integer>	each DFA state = set of NFA states	sorted + no duplicates, so {q0,q1} and {q1,q0} are the same set
NFA→DFA	HashMap<Set, Integer>	set -> DFA state number	O(1) "have I seen this set before?"
NFA→DFA	Queue (LinkedList)	BFS over new subsets	discovers DFA states level by level, no recursion
NFA→DFA	ArrayDeque as stack	ε-closure DFS	pop a state, push its epsilon-targets
Minimize	boolean[][] marked	table-filling pair table	marked[i][j] = i and j are distinguishable
Minimize	Queue + int[] group	BFS reachability; old state -> merged group	delete unreachable states, then merge equivalents
Intersect	static class DFA	groups n, delta, start, isFinal	one object = one whole machine
Intersect	ArrayList<int[]> + HashMap<String, Integer>	reachable pairs (p,q)	product construction by BFS; key "p,q" -> pair number
PDA	static class Rule / Config	one transition rule / one snapshot	state + input position + stack in one object
PDA	String as the stack	top = charAt(0)	push = push + rest, pop = substring(1)
PDA	Queue + HashSet<String> seen	BFS over configurations	tries ALL non-deterministic moves; seen stops infinite ε-loops
PDA	Config.parent pointer	backwards linked list	walk back to rebuild the accepting path, then Collections.reverse

B3. One-liners if asked "what is X?"

Structure	Answer
Array	fixed size, same type, access by index in O(1) - delta[2][1]
2-D array	array of arrays = a table/matrix; that IS the transition table
ArrayList	growable array - used when the size is unknown in advance (list of DFA states)
LinkedList (as Queue)	add to tail, poll from head - used for BFS everywhere
Queue (FIFO)	First In First Out - BFS in subset construction, minimization, product, PDA
Stack (LIFO)	Last In First Out - DFS in ε-closure (ArrayDeque push/pop); the real PDA stack is the String
HashMap<K,V>	key -> value with O(1) average lookup: set->number, pair->number
HashSet	O(1) "contains?" - NFA final states, visited PDA configurations
TreeSet	HashSet + always sorted - needed so sets compare equal regardless of insertion order
String / StringBuilder	String is immutable (cannot be changed); StringBuilder is changeable

Array vs ArrayList? Array: fixed size, primitives allowed (int), faster. ArrayList: grows automatically, holds objects (Integer), has methods like add and contains.

Why int[][] for DFA but List<Integer>[][] for NFA? DFA: exactly ONE next state per (state, symbol), so one int is enough. NFA: zero or many next states, so each cell must hold a list.

B4. Core Java rapid viva

Q1. JDK vs JRE vs JVM?

JDK = tools to write AND run programs (includes javac). JRE = only run. JVM = the engine that executes bytecode. JDK contains JRE, JRE contains JVM.

Q2. javac vs java?

javac compiles .java source into .class bytecode. java runs that class on the JVM.

Q3. Why must the file name match the class name?

Java rule: public class DFASimulation must live in DFASimulation.java, otherwise it is a compile error.

Q4. Class vs object?

Class = blueprint (DFASimulation). Object = one real instance made with new, e.g. new Scanner(System.in). DFA, Rule and Config are blueprints; I create objects of them.

Q5. Primitive vs wrapper?

int, char, boolean, double are primitives (no methods). Integer, Character, Boolean are wrapper objects - collections cannot hold primitives, so List<Integer> not List<int>. Java converts automatically: autoboxing (int -> Integer) and unboxing.

Q6. What does new do? What is a constructor?

new allocates memory on the heap and runs the constructor. A constructor has the same name as the class and no return type, e.g. Rule(int from, char input, ...). this.from = from means "this current object's field" when the parameter has the same name.

Q7. Scanner: next() vs nextLine() vs nextInt()?

nextInt() reads a number, next() reads ONE word, nextLine() reads the WHOLE line. After nextInt() the newline is left in the buffer, so I call sc.nextLine() once to clear it before reading strings.

Q8. == vs .equals()?

== compares references (same memory) for objects; .equals() compares content - w.equalsIgnoreCase("exit") compares text ignoring case. For primitives like char, == compares values, which is why r.top != top is correct.

Q9. Why StringBuilder?

String is immutable - every + creates a NEW string object, slow in loops. StringBuilder changes one buffer: sb.append(...), then sb.toString().

Q10. printf and %-8s?

Formatted printing. %s = string placeholder, '-' = left align, 8 = column width - used to print the transition tables in neat columns.

Q11. Loops used?

for when the count is known (reading n states), while when unknown (while(!queue.isEmpty()) BFS, while(true) input loop), and the enhanced for (for (Rule r : rules)) over collections.

Q12. break vs continue?

break exits the loop completely; continue skips to the next iteration.

Q13. Ternary operator?

condition ? a : b - a one-line if-else. (isFinal[i] ? "*" : " ") prints * for final states.

Q14. What is generics <Integer>?

Type safety for collections: the list may ONLY hold Integers, so no casting errors later.

Q15. Access modifiers?

public = everywhere, private = own class only, protected = package + subclasses, default = same package.

Q16. static nested class?

A class inside another class. Being static, it can be used (new DFA(), new Rule(...)) without an object of the outer class, and it needs no separate file.

Q17. Recursion anywhere?

Avoided on purpose: an explicit stack/queue (ArrayDeque for ε-closure, queues for BFS) gives the same traversal and cannot overflow the call stack on big inputs.

Q18. Exceptions during a demo?

InputMismatchException = text where a number was expected. ArrayIndexOutOfBoundsException = state number ≥ n. NoSuchElementException = the piped demo input finished - just means the run ended.

B5. "Explain your program" - 20-second answers

Q1. DFA Simulation

I store the transition function as a 2-D int array, delta[state][symbol] = next state. Starting at q0, every character is one table lookup. At the end, if the state is final (boolean array check) the string is accepted. Complexity O(length of string).

Q2. NFA to DFA

Subset construction. Each DFA state is a SET of NFA states kept in a TreeSet so order does not matter; a HashMap maps each set to its DFA number; a queue does BFS over new sets. ε-closure uses a stack-based DFS. A DFA state is final if it contains any NFA final state.

Q3. DFA Minimization

Three steps: BFS from the start deletes unreachable states; table filling marks pairs (final, non-final) first, then keeps marking (p,q) if their moves reach an already-marked pair; unmarked pairs are equivalent and get merged. The result is the unique minimal DFA.

Q4. Intersection

Product construction. Each new state is a pair (p,q), p from DFA1, q from DFA2; on symbol a it moves to (δ1(p,a), δ2(q,a)). A pair is final only if BOTH are final. BFS builds only reachable pairs. This proves regular languages are closed under intersection.

Q5. PDA

The stack is a String with the top at index 0; push = prefix, pop = substring(1). Because the PDA is non-deterministic, I run BFS over configurations (state, input position, stack). A HashSet of visited configurations stops infinite ε-loops. Each Config stores a parent pointer, so when one accepts I walk back and print the full accepting path.

B6. Rapid fire - memorize these 10

#	One-liner
1	static = belongs to the class, shared, no object needed
2	main is static so the JVM can call it without creating an object
3	DFA table = int[][]; NFA table = List[][] because an NFA has SETS of next states
4	Queue = FIFO -> BFS. Stack = LIFO -> DFS (my ε-closure)
5	TreeSet = sorted set; HashMap = key->value in O(1); HashSet = fast "contains?"
6	String is immutable -> use StringBuilder for building text in loops
7	== compares references, .equals() compares content

#	One-liner
8	javac compiles to bytecode, java runs it on the JVM
9	Collections hold objects only -> List<Integer>, autoboxing converts int <-> Integer
10	String[] args in main = command-line arguments

PART C - AUTOMATA THEORY VIVA (68 questions, Basic -> Advanced)

Level 1 - Absolute basics (they WILL ask these)

Q1. What is an automaton?

An abstract machine that reads an input string symbol by symbol, changes state according to fixed rules, and finally says accept or reject.

Q2. What is an alphabet (Σ)?

A finite, non-empty set of symbols, e.g. Σ = {a, b} or {0, 1}.

Q3. What is a string? What is ε?

A finite sequence of symbols from Σ. ε (epsilon) is the empty string - length 0.

Q4. What is a language?

A set of strings over Σ, e.g. L = { strings ending in "ab" }.

Q5. What is Σ*?

The set of ALL strings over Σ, including ε. Every language over Σ is a subset of Σ*.

Q6. What are the 5 tuples of a DFA?

M = (Q, Σ, δ, q0, F): Q = finite set of states, Σ = input alphabet, δ : Q × Σ -> Q = transition function, q0 = start state, F ⊆ Q = final states.

Q7. What does "deterministic" mean?

For every state and every symbol there is exactly ONE next state. No choices, no ε-moves.

Q8. When is a string accepted by a DFA?

After reading the whole string, if the machine is in a final state -> accepted, otherwise rejected.

Q9. What is a dead / trap state?

A non-final state that goes to itself on every symbol; once entered, the string can never be accepted.

Q10. What is a regular language?

A language that some DFA (equivalently NFA or regular expression) accepts.

Level 2 - About each program

Program 1 - DFA Simulation

Q11. How did you store the DFA in Java?

As a 2-D int array delta[state][symbolIndex]. Row = current state, column = symbol, value = next state. Final states are a boolean[] isFinal.

Q12. Explain the simulation algorithm.

current = start; for each character find its column, then current = delta[current][column]. At the end check isFinal[current].

Q13. Time complexity?

O(n) where n = length of the input string - each symbol is one O(1) array lookup.

Q14. What if the input has a symbol not in Σ?

The program prints a message and rejects, because δ is not defined for that symbol.

Q15. How is the empty string handled?

User types eps; the loop runs zero times, so acceptance depends only on whether the START state is final.

Q16. What is δ* (extended transition function)?

δ*(q, w) = state reached from q after reading all of w. δ*(q, ε) = q and δ*(q, wa) = δ(δ*(q, w), a). My loop is exactly this definition.

Program 2 - NFA to DFA

Q17. 5 tuples of an NFA? How is δ different?

Same 5 tuples, but δ : Q × (Σ ∪ {ε}) -> 2^Q (power set). One state + one symbol can go to zero, one or many states.

Q18. When does an NFA accept?

If AT LEAST ONE possible path ends in a final state after reading the whole input.

Q19. Are NFAs more powerful than DFAs?

No - they accept exactly the same class, the regular languages. NFAs are just easier to design; that is why subset construction always works.

Q20. What is subset construction?

Each DFA state = a SET of NFA states the machine could be in at once. Start = ε-closure(q0). For each DFA state S and symbol a: next = ε-closure(∪ δ(q,a) for q in S). Repeat until no new set appears.

Q21. What is ε-closure?

All states reachable using only ε-moves (including the state itself). My closure() uses a stack (DFS) to find it.

Q22. Which DFA states are final?

Any set containing at least one NFA final state.

Q23. Maximum number of DFA states?

2^n for n NFA states. My program builds only the REACHABLE subsets (queue/BFS), so usually far fewer.

Q24. Why does the empty set {} appear?

When no NFA state has a move on some symbol; {} becomes the dead state of the DFA.

Q25. Why TreeSet and HashMap?

TreeSet keeps states sorted so {q1,q0} and {q0,q1} are the same set. HashMap gives each set its DFA number in O(1) to check "seen this set before?".

Program 3 - DFA Minimization

Q26. Why minimize a DFA?

Same language with fewer states -> less memory and faster execution; the minimal DFA is unique, so it can also be used to test equivalence of two DFAs.

Q27. What are equivalent states?

p and q are equivalent if for EVERY string w, δ*(p,w) and δ*(q,w) are both final or both non-final.

Q28. What are distinguishable states?

There exists at least one string w where one ends in a final state and the other does not.

Q29. Explain your steps.

1) Remove unreachable states (BFS from start). 2) Mark every (final, non-final) pair - ε distinguishes them. 3) Repeat: mark (p,q) if for some symbol a the pair (δ(p,a), δ(q,a)) is already marked. 4) Stop when a full pass marks nothing new. 5) Unmarked pairs are equivalent - merge them.

Q30. What is this method called?

Table-filling method (based on Myhill-Nerode). The other popular one is the partition / equivalence method (0-equivalence, 1-equivalence, ...).

Q31. Why remove unreachable states first?

They never affect the language, but they could stop the result from being minimal.

Q32. Complexity?

Simple version O(n^2 · |Σ| · n) worst case (n^2 pairs, up to n passes); Hopcroft's algorithm does O(n · |Σ| · log n).

Q33. Is the minimal DFA unique?

Yes, unique up to renaming of states - Myhill-Nerode theorem.

Program 4 - Intersection

Q34. What is product construction?

Run both DFAs in parallel. New state = pair (p,q); δ((p,q), a) = (δ1(p,a), δ2(q,a)); start = (start1, start2).

Q35. Final states for intersection?

(p,q) is final iff p ∈ F1 AND q ∈ F2.

Q36. Change it for union? difference?

Union: p ∈ F1 OR q ∈ F2. Difference L1 - L2: p ∈ F1 AND q ∉ F2. Only the final-state rule changes.

Q37. How many states can the product have?

At most |Q1| × |Q2|; my program builds only reachable pairs (BFS).

Q38. Which closure property does this prove?

Regular languages are closed under intersection.

Q39. Why must both DFAs have the same alphabet?

Both machines read the same symbol at the same time. If alphabets differ, take the union of alphabets and add dead states.

Program 5 - PDA Simulation

Q40. Why do we need a PDA?

A DFA has finite memory and cannot count without limit, so L = {$a^n b^n$} is not regular. A PDA adds a STACK, so it can count and match.

Q41. 7 tuples of a PDA?

P = (Q, Σ, Γ, δ, q0, Z0, F): Γ = stack alphabet, Z0 = initial stack symbol, δ : Q × (Σ ∪ {ε}) × Γ -> finite subsets of Q × Γ*.

Q42. What is an instantaneous description (ID)?

(current state, remaining input, stack contents). My Config class stores exactly this.

Q43. What are the stack operations?

Push: δ(q,a,A) = (p, BA). Pop: δ(q,a,A) = (p, ε). No change: δ(q,a,A) = (p, A).

Q44. Two ways a PDA can accept?

By final state (input finished AND in a final state) or by empty stack (input finished AND stack empty). Equally powerful.

Q45. How does your program handle non-determinism?

It tries ALL possible moves using BFS over configurations; if any branch accepts, the string is accepted. Parent pointers print the accepting path.

Q46. Why a stack limit and a seen set?

ε-moves that keep pushing could run forever. seen stops revisiting the same configuration; the stack limit cuts off branches that grow too big.

Q47. Is a DPDA as powerful as an NPDA?

No - NPDA is strictly more powerful. {ww^R} (even palindromes) needs guessing the middle, so it needs non-determinism.

Q48. What languages do PDAs accept?

Context-free languages (CFLs) - the same ones generated by context-free grammars.

Q49. Design a PDA for $a^n b^n$ (n ≥ 1) by empty stack.

δ(q0,a,Z) = (q0,AZ); δ(q0,a,A) = (q0,AA); δ(q0,b,A) = (q1,ε); δ(q1,b,A) = (q1,ε); δ(q1,ε,Z) = (q1,ε).

Level 3 - Advanced / tricky

Q50. Can a DFA have more than one start state? An NFA?

DFA - no, exactly one. Standard NFA - one; multiple start states can be simulated with one new start state plus ε-moves.

Q51. Can a DFA have zero final states?

Yes - it then accepts the empty language ∅.

Q52. Is the complement of a regular language regular?

Yes - swap final and non-final states of a COMPLETE DFA (does not work directly on an NFA).

Q53. Are CFLs closed under intersection?

No: {$a^n b^n c^m$} ∩ {$a^m b^n c^n$} = {$a^n b^n c^n$} is not context-free. But CFL ∩ Regular is always context-free.

Q54. What is the Pumping Lemma for?

Proving a language is NOT regular (or not context-free), e.g. $a^n b^n$ is not regular.

Q55. What is the Myhill-Nerode theorem?

L is regular iff the relation "x ≡ y iff for all z, xz ∈ L ⇔ yz ∈ L" has finitely many equivalence classes; the number of classes = number of states of the minimal DFA.

Q56. Check if two DFAs accept the same language?

Minimize both and compare (identical up to renaming), or build the product for (L1 - L2) ∪ (L2 - L1) and check that no final state is reachable.

Q57. Chomsky hierarchy - where do these machines fit?

Type 3 Regular -> DFA/NFA; Type 2 Context-free -> PDA; Type 1 Context-sensitive -> LBA; Type 0 Recursively enumerable -> Turing machine.

Q58. Difference between a PDA and a Turing machine?

A PDA only accesses the TOP of its stack and reads input once left to right; a TM has an infinite tape it can read/write and move both ways - much more powerful.

Q59. Why does NFA->DFA use a queue and not recursion?

A queue gives BFS, so DFA states are numbered in discovery order, and there is no risk of stack overflow for large NFAs.

Q60. Real-world uses?

DFA: lexical analysis in compilers, grep pattern matching, protocol checkers, vending machines. PDA: parsing / syntax analysis, balanced-bracket checking, XML/HTML validation.

PART D - LAST-MINUTE CHEAT SHEET

```
cd "C:\Users\bsain\Desktop\AI\SEM 5\FLA\FLA_Project\for_submission"

javac *.java

java DFASimulation   java NFAtoDFA   java DFAMinimization

java DFAIntersection java PDASimulation
```

The 5 programs in one line each

Program	One-liner
DFASimulation	int[][] delta table; start at q0, one lookup per symbol, final state at the end = accept
NFAtoDFA	each DFA state = TreeSet of NFA states; HashMap gives its number; queue/BFS; ε-closure by stack DFS
DFAMinimization	BFS removes unreachable states; table-filling marks distinguishable pairs; unmarked = merge
DFAIntersection	product construction: states are pairs (p,q); final iff BOTH final; BFS keeps reachable pairs
PDASimulation	stack = String (top at index 0); BFS over configurations handles non-determinism; parent pointers print the path

If sir says "explain static"...

Belongs to the class, not an object - one shared copy, usable without new. My static variables: n, symbols, rules. My static methods: main, closure, simulate, printTable. main is static so the JVM can start the program without creating an object.

If sir says "what data structures?"...

Arrays for the tables (int[][] delta, boolean[] isFinal, boolean[][] marked), List<Integer>[][] for the NFA because a cell is a SET of states, TreeSet + HashMap for subset construction, Queue for BFS, ArrayDeque as a stack for ε-closure, a String as the PDA stack, StringBuilder for text in loops.

Numbers worth remembering

Fact	Value
DFA tuples	5: (Q, Σ, δ, q0, F)
PDA tuples	7: (Q, Σ, Γ, δ, q0, Z0, F)
DFA simulation complexity	O(length of string)
Worst-case DFA states after NFA->DFA	2^n (my program builds only reachable ones)
Product construction max states	|Q1| × |Q2|
NFA vs DFA power	equal - both accept exactly the regular languages
DPDA vs NPDA power	NPDA strictly more powerful (ww^R needs non-determinism)
PDA accepts	context-free languages
Minimal DFA	unique up to renaming (Myhill-Nerode)
$a^n b^n$	context-free, NOT regular (DFA cannot count unbounded n)